



VELS



INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)

(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)

PALLAVARAM, THIALAMBUR, PERIYAPALAYAM-CHENNAI

ACCREDITED BY **NAAC** WITH '**A++**' GRADE

SCHOOL OF COMPUTING SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

24PMIT31-DevSec Ops Tools and Techniques Lab

RECORD NOTE BOOK

M.Sc., Information Technology

SECOND YEAR – III SEMESTER

2025 - 2026

NAME : _____

REG NO : _____

NOVEMBER 2025



VELS



INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)

(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)

PALLAVARAM, THIALAMBUR, PERIYAPALAYAM-CHENNAI

ACCREDITED BY NAAC WITH 'A++' GRADE

SCHOOL OF COMPUTING SCIENCES

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY**



CERTIFICATE

*Certified that this is a bonafide record of practical work done by
Mr. /Ms. _____ Reg.No. _____ in the M.Sc.,
(Information Technology) Laboratory during the Year 2025-2026.*

STAFF IN-CHARGE

HEAD OF THE DEPARTMENT

*Submitted for M.Sc., Degree practical examination held on _____ in the
Department of Computer Science and Information Technology at VISTAS.*

DATE:

Examiners

PLACE: Chennai

1.

2.

INDEX

S.NO.	DATE	CONTENTS	PAGE NO.	SIGNATURE
1.		GIT AND GIT REPOSITORY HOSTING PLATFORM		
2.		CONTINUOUS INTEGRATION SETUP USING JENKINS		
3.		IMPLEMENT STATIC APPLICATION SECURITY TESTING (SAST) IN THE CI/CD PIPELINE		
4.		CONTAINERIZE AN APPLICATION USING DOCKER		
5.		DEPLOY AND MANAGE A CONTAINERIZED APPLICATION USING KUBERNETES		
6.		AUTOMATE INFRASTRUCTURE PROVISIONING USING TERRAFORM		
7.		SET UP MONITORING FOR YOUR APPLICATION USING PROMETHEUS AND GRAFANA		
8.		SET UP A SECURE DEVELOPMENT ENVIRONMENT USING GIT		
9.		INTEGRATE SOFTWARE COMPOSITION ANALYSIS (SCA) INTO THE CI/CD PIPELINE		
10.		INTEGRATE CONTAINER SECURITY INTO CI/CD PIPELINE		
11.		IMPLEMENT RUNTIME APPLICATION SELF-PROTECTION (RASP) TO DEFEND APPLICATIONS FROM ATTACKS		
12.		SET UP CONTINUOUS MONITORING OF APPLICATION AND INFRASTRUCTURE AND PRACTICE INCIDENT RESPONSE		
13.		IMPLEMENT SECURITY MONITORING USING CLOUD-NATIVE TOOLS(AWS GUARDDUTY OR AZURE SECURITY CENTER)		

Exercise 1 Version Control with Git and GitHub Repository Hosting Platform

Aim:

To understand the basics of version control, branching, and collaboration using Git and a Git repository hosting platform (GitHub or GitLab).

Procedure:

Step 1: Configure Git

Step 2: Initialize a Local Repository

Step 3: Add and Commit Files

Step 4: Create and Work with Branches

Step 5: Connect to Remote Repository (GitHub or GitLab)

Step 6: Collaborate and Pull Changes

Coding:

Configure Git

1. Open Git Bash or Terminal.
2. Set up user information:

```
git config --global user.name "Your Name"  
git config --global user.email "your_email@example.com"
```

3. Verify configuration:

```
git config --list
```

Initialize a Local Repository

1. Create a new folder:

```
mkdir version-control-lab  
cd version-control-lab
```

2. Initialize a Git repository:

```
git init
```

Output: Initialized empty Git repository in...

Add and Commit Files

1. Create a new file:

```
echo "Hello Git World!" > hello.txt
```

2. Check the repository status:

```
git status
```

3. Add the file to the staging area:

```
git add hello.txt
```

4. Commit the file:

```
git commit -m "Initial commit: added hello.txt"
```

5. View commit history:

```
git log --oneline
```

Create and Work with Branches

1. Create a new branch:

```
git branch feature-update
```

2. Switch to the new branch:

```
git checkout feature-update
```

3. Modify the file:

```
echo "Added more content in the feature branch." >> hello.txt
```

4. Add and commit the change:

```
git add hello.txt
```

```
git commit -m "Updated hello.txt with new content"
```

5. Switch back to the main branch:

```
git checkout main
```

6. Merge the feature branch:

```
git merge feature-update
```

7. Display file content:

```
cat hello.txt
```

Connect to Remote Repository (GitHub or GitLab)

1. Log in to GitHub or GitLab.
2. Create a new repository (e.g., version-control-lab).
3. Copy the repository URL (HTTPS).
4. Add the remote repository:

```
git remote add origin https://github.com/<username>/version-control-lab.git
```

5. Push local commits to remote:

```
git push -u origin main
```

Collaborate and Pull Changes

1. A collaborator clones your repository:

```
git clone https://github.com/<username>/version-control-lab.git
```

2. They make changes, commit, and push to their fork or open a merge request.
3. You can pull updates into your local repository:

```
git pull origin main
```

4. Review changes:

```
git log  
git diff
```

Output:

Initialized empty Git repository in /home/user/version-control-lab/.git/

[main (root-commit) 9a3c4e2] Initial commit: added hello.txt

1 file changed, 1 insertion(+)

Switched to branch 'feature-update'

Merge made by the 'recursive' strategy.

Result:

Successfully understood and implemented version control basics including repository initialization, staging, committing, branching, merging, pushing, and pulling using Git and GitHub/GitLab.

Exercise 2

Continuous Integration Setup using Jenkins

Aim:

To set up and configure Jenkins for Continuous Integration (CI) to automate the process of building and testing an application.

Procedure:

Step 1: Install Java

Step 2: Install Jenkins on Windows

Step 3: Start Jenkins Service

Step 4: Unlock Jenkins

Step 5: Create a New Jenkins Job

Step 6: Configure Source Code Management

Step 7: Configure Build Triggers

Step 8: Execute the Pipeline

Step 9: Verify Continuous Integration

Coding:

1. Install Java on Windows
2. Install Jenkins on Windows
 - Download JDK from Oracle or OpenJDK.
 - Download Jenkins from <https://www.jenkins.io/download/>.
 - Install using default settings and start the Jenkins service.
 - Open a web browser and go to:
<http://localhost:8080>

Unlock Jenkins

1. Retrieve the initial admin password:

```
cat /var/lib/jenkins/secrets/initialAdminPassword
```

2. Copy the password and paste it into the setup screen.
3. Install **Suggested Plugins**.
4. Create an **admin user** and complete the setup.

Create a New Jenkins Job

1. On the Jenkins Dashboard, click “**New Item**”.
2. Enter a job name (e.g., MyFirstCIPProject).
3. Select **Freestyle Project** and click **OK**.

Configure Source Code Management

1. Under **Source Code Management**, select **Git**.
2. Enter your Git repository URL, for example:

```
https://github.com/username/sample-app.git
```

3. Add credentials if the repository is private.

Configure Build Triggers

Enable Jenkins to automatically run builds when code changes are committed.
Choose:

- **Poll SCM** (for periodic checks) or
- **GitHub hook trigger for GITScm polling**

Polling Schedule (every minute):

* * * * *

Execute the Pipeline

1. Click **Build Now** in Jenkins Dashboard.
2. Observe build progress under **Build History**.
3. Click the build number → **Console Output** to view build logs.

Output:

Building project...
Tests completed successfully.
Build finished: SUCCESS

Verify Continuous Integration

- Commit new code to GitHub.
- Jenkins will automatically detect the change, build, and test the project again.

Output:

Jenkins Dashboard:

- Job: MyFirstCIProject
- Build Status: SUCCESS
- Stages: Git → Build → Test

Result:

Successfully set up Jenkins for Continuous Integration, automating the build and test process of an application.

Exercise 3: Implement Static Application Security Testing (SAST) in CI/CD Pipeline

Aim:

To implement Static Application Security Testing (SAST) in a GitLab CI/CD pipeline to automatically detect and fix vulnerabilities during the software build, test, and deployment process.

Procedure:

Step 1: Create a New GitLab Project

Step 2: Clone the Repository

Step 3: Add Sample Application Code

Step 4: Create the GitLab CI/CD Configuration File

Step 5: Commit and Push Pipeline Configuration

Step 6: Run the Pipeline

Step 7: View the SAST Report

Coding:

Create a New GitLab Project

1. Log in to your GitLab account at <https://gitlab.com>.
2. Click New Project → Create Blank Project.
3. Enter project name: sast-ci-demo.
4. Initialize the repository with a README.md.

Clone the Repository

Open a terminal and run:

```
git clone https://gitlab.com/<your-username>/sast-ci-demo.git
cd sast-ci-demo
```

Add Sample Application Code

Create a small demo file for testing:

app.py

```
def insecure_code():
    password = "admin123" # Hardcoded password for testing
    print("This is a sample insecure code snippet.")

if __name__ == "__main__":
    insecure_code()
```

requirements.txt

```
flask
```

Save the files, then commit and push:

```
git add .
git commit -m "Added sample Python app"
git push origin main
```

Create the GitLab CI/CD Configuration File

Create a file named **.gitlab-ci.yml** in the root of the repository.

Program: .gitlab-ci.yml

stages:

- test
- sast
- deploy

variables:

DOCKER_DRIVER: overlay2

1. Unit Testing Stage

unit_test:

stage: test

image: python:3.9

script:

- pip install -r requirements.txt
- python -m unittest discover

artifacts:

when: always

reports:

junit: test-results.xml

only:

- main

2. Include GitLab's Predefined SAST Template

include:

- template: Security/SAST.gitlab-ci.yml

sast:

stage: sast

allow_failure: false

only:

- main

3. Deployment Stage (Simulated)

deploy_app:

stage: deploy

script:

- echo "Deploying application..."
- echo "Application deployed successfully!"

only:

- main

Commit and Push Pipeline Configuration

```
git add .gitlab-ci.yml
git commit -m "Added GitLab CI/CD pipeline with SAST scanning"
git push origin main
```

Run the Pipeline

1. Go to your GitLab project → CI/CD → Pipelines.
2. Observe three stages:
 - test
 - sast
 - deploy
3. Click each job to view its Console Output logs.

View the SAST Report

1. Navigate to Security & Compliance → Vulnerability Report in your GitLab project.
2. You will see any vulnerabilities detected in your source code (for example, hardcoded passwords or insecure imports).
3. Click each issue to see detailed remediation advice.

Output:

Pipeline Output (Stages):

Stage	Status	Description
Test	✓ Passed	Unit tests executed successfully
SAST	⚠ Issues Found	Detected insecure code pattern
Deploy	✓ Passed	Application deployed successfully

Result:

Successfully configured and executed a GitLab CI/CD pipeline integrated with Static Application Security Testing (SAST) to automatically scan, detect, and report security vulnerabilities before deployment.

Exercise 4: Containerize an application using Docker

Aim

To understand and implement the process of containerizing an application using Docker by creating, building, and running containers for a sample application.

Procedure

Step 1: Verify Docker Installation

Step 2: Create a Sample Application

Step 3: Write a Dockerfile

Step 4: Build the Docker Image

Step 5: Verify the Docker Image

Step 6: Run the Container

Step 7: Test the Application

Step 8: View Running Containers

Step 9: Stop and Remove the Container

Step 10: Push Image to Docker Hub

Coding:

Verify Docker Installation

Check Docker version:

```
docker --version
```

Expected Output:

Docker version 27.0.1, build 123456

Create a Sample Application

Example: **Python Flask Application**

Create a directory:

```
mkdir docker-lab  
cd docker-lab
```

Create a file named **app.py**:

```
from flask import Flask  
  
app = Flask(__name__)  
  
@app.route('/')  
def home():  
    return "Hello from Dockerized Flask App!"  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```

Create a **requirements.txt** file:

```
flask
```

Write a Dockerfile

Create a file named **Dockerfile** in the same folder:

```
# Step 1: Use an official Python base image  
FROM python: 3.9-slim
```

Step 2: Set working directory

WORKDIR /app

Step 3: Copy project files into the container

COPY . /app

Step 4: Install dependencies

RUN pip install -r requirements.txt

Step 5: Expose port 5000

EXPOSE 5000

Step 6: Define the command to run the app

CMD ["python", "app.py"]

Build the Docker Image

Run the following command:

```
docker build -t flask-app:latest .
```

Verify the Docker Image

```
docker images
```

Output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
flask-app	latest	45efc34abc12	30 seconds ago	125MB

Run the Container

```
docker run -d -p 5000:5000 flask-app
```

Test the Application

Open a browser and navigate to:

<http://localhost:5000>

Output:

Hello from Dockerized Flask App!

View Running Containers

```
docker ps
```

Stop and Remove the Container

```
docker stop <container_id>
```

```
docker rm <container_id>
```

Push Image to Docker Hub (Optional)

1. Login to Docker Hub:
2. `docker login`
3. Tag the image:
4. `docker tag flask-app <your-username>/flask-app:v1`
5. Push the image:
6. `docker push <your-username>/flask-app:v1`

Output:**Terminal Output:**

```
$ docker build -t flask-app .
```

```
Successfully built flask-app
```

```
$ docker run -d -p 5000:5000 flask-app
```

```
Container started...
```

Browser Output:

Hello from Dockerized Flask App!

Docker Dashboard Output:

- Image: flask-app:latest
- Container Status: Running

Result:

Successfully containerized a sample Python Flask application using Docker by creating an image, running it as a container, and verifying its functionality through a web browser.

Exercise 5: Deploy and manage a containerized application using Kubernetes

Aim:

To understand the process of deploying and managing a containerized application using Kubernetes.

Procedure

Step 1: Start Kubernetes Cluster

Step 2: Create a Sample Application

Step 3: Create Dockerfile

Step 4: Create Kubernetes Deployment Manifest

Step 5: Create Kubernetes Service Manifest

Step 6: Deploy Application to Kubernetes

Step 7: Access the Application

Step 8: Scale the Application

Step 9: Delete Resources

Coding:

Start Kubernetes Cluster

If using **Minikube** (for local setup):

```
minikube start
```

Verify:

```
kubectl cluster-info
```

Output:

Kubernetes control plane is running at https://127.0.0.1:8443

Create a Sample Application

Example: Python Flask Application

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Kubernetes Flask App!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
flask
```

Create Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY . /app
RUN pip install -r requirements.txt
```

EXPOSE 5000

CMD ["python", "app.py"]

Build and push the image to Docker Hub:

docker build -t <your-dockerhub-username>/kube-flask:latest .

docker login

docker push <your-dockerhub-username>/kube-flask:latest

Create Kubernetes Deployment Manifest

Create a file **deployment.yaml**:

apiVersion: apps/v1

kind: Deployment

metadata:

name: flask-deployment

spec:

replicas: 2

selector:

matchLabels:

app: flask-app

template:

metadata:

labels:

app: flask-app

spec:

containers:

- name: flask-container

image: <your-dockerhub-username>/kube-flask:latest

ports:

- containerPort: 5000

Create Kubernetes Service Manifest

Create a file **service.yaml**:

apiVersion: v1

kind: Service

metadata:

name: flask-service

spec:

type: NodePort

```
selector:
  app: flask-app
ports:
  - port: 5000
    targetPort: 5000
    nodePort: 30001
```

Deploy Application to Kubernetes

Apply configuration files:

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

Verify running resources:

```
kubectl get pods
kubectl get deployments
kubectl get svc
```

Output:

NAME	READY	STATUS	RESTARTS	AGE
flask-deployment	2/2	Running	0	2m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
flask-service	NodePort	10.108.0.1	<none>	5000:30001/TCP	2m

Access the Application

If using **Minikube**, run:

```
minikube service flask-service
```

This will open the browser automatically.

Alternatively, access:

<http://localhost:30001>

Output:

Hello from Kubernetes Flask App!

Scale the Application

Increase replicas to 4:

```
kubectl scale deployment flask-deployment --replicas=4  
kubectl get pods
```

4 Pods running.

Delete Resources

To clean up:

```
kubectl delete -f deployment.yaml  
kubectl delete -f service.yaml
```

Output:

Terminal Output:

```
deployment.apps/flask-deployment created  
service/flask-service created
```

Browser Output:

Hello from Kubernetes Flask App!

Pods Output:

```
kubectl get pods
```

NAME	READY	STATUS	AGE
flask-deployment-xyz123	1/1	Running	3m
flask-deployment-abc456	1/1	Running	3m

Result:

Successfully deployed and managed a containerized Python Flask application on a Kubernetes cluster, using Deployments and Services to ensure availability, scalability, and accessibility.

Exercise 6: Automate infrastructure provisioning using Terraform

Aim

To understand and implement Infrastructure as Code (IaC) using Terraform to automate the creation, modification, and destruction of cloud infrastructure resources.

Procedure

Step 1: Install and Verify Terraform

Step 2: Configure Cloud Provider (Example: AWS)

Step 3: Create a Working Directory

Step 4: Create Terraform Configuration File

Step 5: Initialize Terraform

Step 6: Validate and Plan the Deployment

Step 7: Apply the Configuration

Step 8: Verify State File

Step 9: Modify or Scale Infrastructure

Step 10: Destroy Infrastructure

Coding:

Install and Verify Terraform

Download Terraform from <https://developer.hashicorp.com/terraform/downloads>

Verify installation:

```
terraform -version
```

Output:

Terraform v1.9.0

Configure Cloud Provider (Example: AWS)

Set up credentials:

```
aws configure
```

Enter:

AWS Access Key ID: <your_access_key>

AWS Secret Access Key: <your_secret_key>

Default region name: ap-south-1

Default output format: json

Create a Working Directory

```
mkdir terraform-lab
```

```
cd terraform-lab
```

Create Terraform Configuration File

Create a file named **main.tf**

Program — main.tf

```
# Configure the AWS Provider
```

```
provider "aws" {
```

```
    region = "ap-south-1"
```

```
}

# Create an AWS EC2 Instance
resource "aws_instance" "lab_server" {
  ami      = "ami-0e742cca61fb65052" # Example Amazon Linux 2 AMI ID (region-
specific)
  instance_type = "t2.micro"
  tags = {
    Name = "TerraformLabInstance"
  }
}
```

Initialize Terraform

Initialize the working directory to download provider plugins:

```
terraform init
```

Output:

Terraform has been successfully initialized!

Validate and Plan the Deployment

Validate the syntax:

```
terraform validate
```

Create an execution plan:

```
terraform plan
```

Output:

Plan: 1 to add, 0 to change, 0 to destroy.

Apply the Configuration

Deploy the infrastructure:

```
terraform apply
```

Type yes when prompted.

Output:

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Check in AWS Console → EC2 → Instances.

Verify State File

Terraform creates a file **terraform.tfstate** that stores the infrastructure state.

```
cat terraform.tfstate
```

Modify or Scale Infrastructure

Update instance type:

```
instance_type = "t3.micro"
```

Then run:

```
terraform apply
```

Terraform will automatically update the instance as per new configuration.

Destroy Infrastructure

To clean up resources:

```
terraform destroy
```

Confirm with yes.

Output:

Destroy complete! Resources: 1 destroyed.

Command-Line Output:

Terraform initialized successfully.

Plan: 1 to add, 0 to change, 0 to destroy.

aws_instance.lab_server: Creation complete after 45s

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

AWS Console Output:

- EC2 Instance running: **TerraformLabInstance**
- Instance Type: t2.micro
- Status: Running

Result:

Successfully automated the provisioning and management of an AWS EC2 instance using Terraform.

Exercise 7: Set up monitoring for your application using Prometheus and Grafana

Aim

To monitor the performance and health of an application by setting up Prometheus for metrics collection and Grafana for visualizing real-time data and creating dashboards.

Procedure:

Step 1: Create a Sample Application

Step 2: Set Up Prometheus

Step 3: Add Grafana

Step 4: Connect Grafana to Prometheus

Step 5: Create a Dashboard

Step 6: Verify Monitoring

Coding:

Create a Sample Application

app.py

```
from flask import Flask
from prometheus_client import Counter, generate_latest

app = Flask(__name__)

REQUEST_COUNT = Counter('app_requests_total', 'Total HTTP Requests')

@app.route('/')
def home():
    REQUEST_COUNT.inc()
    return "Hello from monitored Flask app!"

@app.route('/metrics')
def metrics():
    return generate_latest()

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Create **requirements.txt**:

```
flask
prometheus_client
```

Run the app:

```
pip install -r requirements.txt
python app.py
```

Visit:

- <http://localhost:5000/> → Home page
- <http://localhost:5000/metrics> → Metrics endpoint

Set up Prometheus

Create **prometheus.yml**:

```
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: 'flask-app'
    static_configs:
      - targets: ['host.docker.internal:5000']
```

Run Prometheus using Docker:

```
docker run -d \
  -p 9090:9090 \
  -v $(pwd)/prometheus.yml:/etc/prometheus/prometheus.yml \
  prom/prometheus
```

Verify Prometheus is running:

Visit: <http://localhost:9090>

Add Grafana

Run Grafana using Docker:

```
docker run -d \
  -p 3000:3000 \
  grafana/grafana
```

Default credentials:

- Username: admin
- Password: admin

Access: <http://localhost:3000>

Connect Grafana to Prometheus

1. Login to Grafana → **Configuration** → **Data Sources** → **Add Data Source**
2. Select **Prometheus**
3. URL: <http://host.docker.internal:9090>
4. Click **Save & Test** → Connection successful

Create a Dashboard

1. In Grafana, click + → **Dashboard** → **Add Panel**
2. Enter query:

```
app_requests_total
```

3. Set visualization type (Graph / Gauge / Table)
4. Save the dashboard

Verify Monitoring

- Visit <http://localhost:3000> → Dashboard shows live updates
- Access <http://localhost:5000/> multiple times → Observe metrics incrementing
- Metrics are updated every 5 seconds (as per Prometheus configuration)

Output:

Prometheus Metrics Endpoint (/metrics):

```
app_requests_total 5.0
```

Result:

Successfully set up Prometheus and Grafana to monitor a sample Flask application. Prometheus collected metrics, and Grafana visualized real-time application performance data.

Exercise: 8 Set up a Secure Development Environment using Git

Aim

To set up a secure development environment that integrates Version Control (Git), Vulnerability scanning (Semgrep, Bandit, Trivy) and Secure coding practices and validation.

Procedure

Step 1: Install and Configure Git

Step 2: Add Secure Coding Example

Step 3: Run SAST (Static Application Security Testing)

Step 4: Fix the Insecure Code

Step 5: Add Dependency Vulnerability Scan (Node.js Example)

Step 6: Container Image Scan (Trivy)

Step 7: Secure Git Commits

Coding:

Install and Configure Git

```
git config --global user.name "student"
```

```
git config --global user.email student@example.com
```

Initialize a repository:

```
mkdir secure-dev-lab && cd secure-dev-lab
```

```
git init
```

```
echo "# Secure Dev Environment" > README.md
```

```
git add README.md
```

```
git commit -m "Initial commit - secure dev setup"
```

Output:

```
Initialized empty Git repository in /home/student/secure-dev-lab/.git/
```

```
[main (root-commit) ab12cd3] Initial commit - secure dev setup
```

```
1 file changed, 1 insertion(+)
```

```
create mode 100644 README.md
```

Add Secure Coding Example

Create a small Python program (app.py) with insecure code

```
# app.py
```

```
import os
```

```
import sqlite3
```

```
def login(username, password):
```

```
    conn = sqlite3.connect('users.db')
```

```
# insecure SQL (for demonstration)

query = f'SELECT * FROM users WHERE username='{username}' AND
password='{password}'

print("Running query:", query)

return conn.execute(query).fetchone()
```

Run SAST (Static Application Security Testing)

Install **Semgrep** and **Bandit**

```
pip install semgrep bandit
```

Run Semgrep:

```
semgrep --config=p/ci
```

Output:

app.py:6: Possible SQL Injection detected

→ Avoid using string concatenation in SQL queries.

Run Bandit:

```
bandit -r app.py
```

Output:

Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector

Severity: HIGH Confidence: HIGH

Location: app.py:6

Fix the Insecure Code

Edit app.py:

```
# Secure version using parameterized queries
import sqlite3

def login(username, password):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
```

```
cursor.execute("SELECT * FROM users WHERE username=? AND password=?",
(username, password))
return cursor.fetchone()
```

Re-run Bandit:

No issues identified.

Add Dependency Vulnerability Scan (Node.js Example)

```
npm init -y
npm install lodash@4.17.19
npm audit
```

Output (truncated):

found 1 high severity vulnerability in lodash 4.17.19
run `npm audit fix` to resolve

Run fix:

```
npm audit fix
```

Output:

```
+ lodash@4.17.21
fixed 1 of 1 vulnerability
```

Container Image Scan (Trivy)

Install and run Trivy:

```
sudo apt install wget -y
wget https://github.com/aquasecurity/trivy/releases/latest/download/trivy_0.50.0_Linux-64bit.deb
sudo dpkg -i trivy_0.50.0_Linux-64bit.deb
```

```
cat > Dockerfile <<'EOF'
FROM python:3.10-slim
COPY app.py /app/app.py
WORKDIR /app
CMD ["python3", "app.py"]
EOF
```

```
docker build -t secure-lab:latest .
trivy image secure-lab:latest
```

Output:

VULNERABILITIES:

```
+-----+-----+-----+
| LIBRARY   | CVE ID     | SEVERITY |
+-----+-----+-----+
| python3   | CVE-2023-#### | MEDIUM  |
+-----+-----+-----+
```

Secure Git Commits

Install pre-commit:

```
pip install pre-commit
cat > .pre-commit-config.yaml <<'EOF'
repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.4.0
  hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer
    - id: check-yaml
EOF
pre-commit install
```

Output:

Trim Trailing Whitespace.....Failed

Output:

Tool	Function	Sample Finding
Bandit	SAST	SQL Injection
Semgrep	SAST	Hardcoded secrets
npm audit	Dependency scan	Lodash vulnerability
Trivy	Container scan	Outdated system packages
Pre-commit	Code hygiene	Trailing whitespace, bad YAML

Result:

A secure development environment was successfully set up with Git version control, Automated static security scanning (SAST), Secure coding validation and Pre-commit and CI-ready scanning tools.

Exercise : 9 Integrate Software Composition Analysis (SCA) into the CI/CD

Aim

To integrate Software Composition Analysis (SCA) into a CI/CD pipeline for automatically detecting vulnerabilities in third-party dependencies during application build and deployment.

Procedure:

Step 1: Create a Sample Project

Step 2: Run Local SCA Scan

Step 3: Create GitHub Repository and Push Code

Step 4: Add CI/CD Pipeline for SCA

Step 5: Add Build Fail Condition

Step 6: Review Reports

Coding:

Create a Sample Project

Java Example:

```
mkdir sca-lab && cd sca-lab
mvn archetype:generate -DgroupId=com.example -DartifactId=sca-demo -
DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
cd sca-demo
```

Node.js Example:

```
mkdir sca-lab && cd sca-lab
npm init -y
npm install express@4.16.0 lodash@4.17.19
```

Output (Node):

added 50 packages in 2s

Run Local SCA Scan

(A) OWASP Dependency-Check (for Java)

Download and run:

```
wget
https://github.com/jeremylong/DependencyCheck/releases/download/v10.0.4/dependency-
check-10.0.4-release.zip
unzip dependency-check-10.0.4-release.zip
./dependency-check/bin/dependency-check.sh --project "SCA Demo" --scan . --format
HTML --out reports/
```

Output (truncated):

Found 2 vulnerabilities:
CVE-2021-44228 (log4j)
Severity: CRITICAL

npm audit (for Node.js)

npm audit

Output:

fixed 2 of 2 vulnerabilities

Create GitHub Repository and Push Code

git init

git add .

git commit -m "Initial commit - add SCA demo"

git branch -M main

git remote add origin https://github.com/<your-username>/sca-lab.git

git push -u origin main

Add CI/CD Pipeline for SCA

Create a file: .github/workflows/sca-pipeline.yml

name: SCA Scan Pipeline

on:

push:

pull_request:

jobs:

sca-scan:

runs-on: ubuntu-latest

steps:

- name: Checkout Repository

uses: actions/checkout@v4

---- Node.js Example ----

- name: Setup Node.js

uses: actions/setup-node@v4

with:

node-version: '18'

- name: Install Dependencies

run: npm ci

- name: Run npm audit

run: |

```
npm audit --json > audit-report.json
cat audit-report.json | jq '.metadata.vulnerabilities'
```

```
- name: Upload SCA Report
  uses: actions/upload-artifact@v4
  with:
    name: npm-audit-report
    path: audit-report.json
```

Output (GitHub Actions console):

Run npm audit

```
{
  "low": 0,
  "moderate": 0,
  "high": 1,
  "critical": 0
}
```

Artifact uploaded: npm-audit-report

Artifact uploaded: dependency-check-report

Add Build Fail Condition

Add this in the pipeline:

```
- name: Fail on high vulnerabilities
  run: |
    count=$(cat audit-report.json | jq '.metadata.vulnerabilities.high')
    if [ "$count" -gt 0 ]; then
      echo "High vulnerabilities found: $count"
      exit 1
    fi
```

Review Reports

From the GitHub Actions “**Artifacts**”, download:

- npm-audit-report.json
- dependency-check-report/index.html

Each shows a detailed list of dependencies, versions, and CVE IDs.

Output:

Tool	Example Vulnerability	Severity	Fixed
npm audit	lodash 4.17.19	High	✓
Dependency-Check	log4j-core 2.14.0	Critical	✓
Trivy	base image python:3.10-slim	Medium	⚠ Pending

Result:

A CI/CD pipeline was successfully configured to perform Software Composition Analysis (SCA) using open-source tools.

Exercise 10: Integrate Container Security into CI/CD pipeline

Aim:

To integrate container security scanning into a CI/CD pipeline to detect vulnerabilities, misconfigurations, and insecure packages in Docker images automatically before deployment.

Procedure:

Step 1: Create a Sample Application

Step 2: Create a Dockerfile

Step 3: Build and Run Docker Image Locally

Step 4: Install and Run Trivy Locally (Optional Test)

Step 5: Push Code to GitHub

Step 6: Add GitHub Actions CI/CD Pipeline

Step 7: Run Pipeline on GitHub

Coding:

Create a Sample Application

Create a simple Python web app:

```
mkdir container-security-lab && cd container-security-lab
cat > app.py <<'EOF'
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Secure Container!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
EOF
```

Create a Dockerfile

```
cat > Dockerfile <<'EOF'
FROM python:3.10-slim
WORKDIR /app
COPY app.py .
RUN pip install flask==2.0.1
EXPOSE 5000
CMD ["python3", "app.py"]
EOF
```

Build and Run Docker Image Locally

```
docker build -t secure-app:latest .
docker run -d -p 5000:5000 secure-app:latest
```

Output:

```
Successfully built a1b2c3d4e5f6
Successfully tagged secure-app:latest
```

Access <http://localhost:5000> → output:

Hello from Secure Container!

Install and Run Trivy Locally (Optional Test)

```
wget https://github.com/aquasecurity/trivy/releases/latest/download/trivy_0.50.0_Linux-64bit.deb
sudo dpkg -i trivy_0.50.0_Linux-64bit.deb
trivy image secure-app:latest
```

Output:

2025-10-21T12:00:00Z INFO Detected OS: debian

VULNERABILITIES:

LIBRARY	VULNERABILITY ID	SEVERITY
flask	CVE-2023-XXXX	HIGH
python3	CVE-2024-YYYY	MEDIUM

Push Code to GitHub

```
git init
git add .
git commit -m "Initial commit - container security lab"
git branch -M main
git remote add origin https://github.com/<your-username>/container-security-lab.git
git push -u origin main
```

Add GitHub Actions CI/CD Pipeline

Create folder `.github/workflows/` and file `container-security.yml`:

name: Container Security Pipeline

on:

push:

pull_request:

jobs:

container-scan:

runs-on: ubuntu-latest

steps:

- name: Checkout Repository

uses: actions/checkout@v4

- name: Set up Docker Buildx
uses: docker/setup-buildx-action@v3
- name: Build Docker Image
run: |
docker build -t secure-app:latest .
- name: Run Trivy Vulnerability Scan
uses: aquasecurity/trivy-action@v0.8.0
with:
image-ref: 'secure-app:latest'
format: 'table'
exit-code: '1'
ignore-unfixed: true
vuln-type: 'os,library'
severity: 'HIGH,CRITICAL'
- name: Upload Trivy Scan Report
uses: actions/upload-artifact@v4
with:
name: trivy-report
path: trivy-results.txt

Run Pipeline on GitHub

1. Push the changes:
2. git add .github/workflows/container-security.yml
3. git commit -m "Add container security pipeline"
4. git push
5. Go to **GitHub** → **Actions** → **Container Security Pipeline**
 - Build Docker image
 - Run Trivy Scan
 - Upload report as artifact
6. **Output (GitHub Actions console):**

```
Detected OS: debian
Total: 3 (HIGH:1, MEDIUM:2)
::error:: Vulnerabilities found: 1 HIGH
Error: Process completed with exit code 1.
```

Result:

A CI/CD pipeline was successfully created to Build Docker images, Scan them for vulnerabilities using Trivy, Optionally check host configuration via Docker Bench, Block deployment if critical vulnerabilities are found.

Exercise 11: Implement runtime application self-protection (RASP) to defend applications from attacks

Aim

To integrate Runtime Application Self-Protection (RASP) into a web application to detect and mitigate malicious inputs and runtime attacks in real time.

Procedure:

Step 1: Create a Sample Vulnerable Application

Step 2: Add a RASP Protection Layer

Step 3: Run the Application

Step 4: Simulate Normal and Malicious Requests

Step 5: Containerize and Test (Optional)

Step 6: Integrate into CI/CD Pipeline

Coding:

Create a Sample Vulnerable Application

```
mkdir rasp-lab && cd rasp-lab
python3 -m venv venv
source venv/bin/activate
pip install flask request
```

Now create a simple web app:

```
# app.py
from flask import Flask, request
import os

app = Flask(__name__)

@app.route('/')
def home():
    return "Welcome to RASP Protected App"

@app.route('/search')
def search():
    query = request.args.get('q')
    # Intentionally insecure query simulation
    return f"Results for: {query}"

@app.route('/run')
def run_command():
    cmd = request.args.get('cmd')
    os.system(cmd) # Vulnerable endpoint
    return "Command executed"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Add a RASP Protection Layer

Create a file rasp_middleware.py:

```
# rasp_middleware.py
from flask import request
import re, sys, logging

logging.basicConfig(filename="rasp.log", level=logging.INFO)

def rasp_protect():
    malicious_patterns = [
        r"(?:)|(?:--)|(/\*(?:.[\n\r])*?\/)|(%27)|(%23)|(%2D%2D)",
        # SQL Injection
        r"(?:<script.*?>.??</script>)",
        # XSS
        r"(?:;|&&|\\|\\)",
        # Command Injection
    ]
    user_input = str(request.args)
    for pattern in malicious_patterns:
        if re.search(pattern, user_input, re.IGNORECASE):
            logging.warning(f"🚫 RASP blocked attack: {user_input}")
            return "🚫 Attack detected and blocked by RASP!", 403
    return None
```

Modify app.py to integrate RASP middleware:

```
# Add this at the top
from rasp_middleware import rasp_protect
from flask import abort

@app.before_request
def before_req():
    blocked = rasp_protect()
    if blocked:
        abort(403)
```

Run the Application

```
python app.py
```

Output:

```
* Running on http://127.0.0.1:5000
```

Simulate Normal and Malicious Requests

Normal request:

```
curl "http://127.0.0.1:5000/search?q=flowers"
```

Output:

```
Results for: flowers
```

Malicious request (SQL Injection):

```
curl "http://127.0.0.1:5000/search?q=' OR 1=1 --"
```

Output:

```
⊘ Attack detected and blocked by RASP!
```

Command injection attempt:

```
curl "http://127.0.0.1:5000/run?cmd=ls;rm -rf /"
```

Output:

```
⊘ Attack detected and blocked by RASP!
```

```
Log file rasp.log:
```

```
WARNING:root:🛡️ RASP blocked attack: {'cmd': ['ls;rm -rf /']}
```

```
WARNING:root:🛡️ RASP blocked attack: {'q': ['" OR 1=1 --"]}
```

Containerize and Test (Optional)

Dockerfile:

```
FROM python:3.10-slim
```

```
WORKDIR /app
COPY . .
RUN pip install flask requests
EXPOSE 5000
CMD ["python", "app.py"]
```

Build and run:

```
docker build -t rasp-app .
docker run -p 5000:5000 rasp-app
```

Integrate into CI/CD Pipeline

Example GitHub Actions step for RASP runtime test:

```
- name: Run RASP App Test
  run: |
    docker run -d -p 5000:5000 rasp-app
    sleep 5
    curl -f "http://localhost:5000/search?q=' OR 1=1 --" && exit 1 || echo "Attack blocked ✓"
```

If RASP works, pipeline continues; if not, build fails.

Output:

Test Type	Input	Expected Result	Status
Normal Input	q=flowers	Page displays results	✓ Passed
SQL Injection	q=' OR 1=1 --	Blocked by RASP	✓ Passed
Command Injection	cmd=ls;rm -rf /	Blocked by RASP	✓ Passed

Result:

The RASP middleware successfully detected and blocked runtime attacks (SQLi, XSS, and command injection) in real time, proving effective self-protection within the application.

Exercise 12: Set up continuous monitoring of application and infrastructure and practice incident response

Aim

To set up a continuous monitoring system for applications and infrastructure using Prometheus and Grafana, and to practice incident detection and response through simulated system failures.

Procedure:

Step 1: Create Project Directory

Step 2: Create a Sample Web Application

Step 3: Dockerize the Application

Step 4: Create Prometheus Configuration

Step 5: Set Up Docker Compose Stack

Step 6: Add Prometheus Data Source in Grafana

Step 7: Create a Grafana Dashboard

Step 8: Configure Alerts in Prometheus

Step 9: Simulate Incident Response

Coding:

Create Project Directory

```
mkdir monitoring-lab && cd monitoring-lab
```

Create a Sample Web Application

Create a simple **Flask** app that exposes a /metrics endpoint for Prometheus.

```
# app.py

from flask import Flask

import time, random

from prometheus_client import Counter, generate_latest, CONTENT_TYPE_LATEST

app = Flask(__name__)

REQUEST_COUNT = Counter('app_requests_total', 'Total number of requests')

@app.route('/')

def home():

    REQUEST_COUNT.inc()

    return "Welcome to the monitored app!"

@app.route('/metrics')

def metrics():

    return generate_latest(), 200, {'Content-Type': CONTENT_TYPE_LATEST}

@app.route('/simulate_error')

def simulate_error():

    time.sleep(random.randint(3, 8)) # artificial delay

    raise Exception("Simulated failure!")
```

```
if __name__ == '__main__':  
  
    app.run(host='0.0.0.0', port=5000)
```

Install dependencies:

```
pip install flask prometheus_client
```

Dockerize the Application

```
# Dockerfile  
FROM python:3.10-slim  
WORKDIR /app  
COPY app.py .  
RUN pip install flask prometheus_client  
EXPOSE 5000  
CMD ["python", "app.py"]
```

Build and run:

```
docker build -t monitored-app .  
docker run -d -p 5000:5000 monitored-app
```

Access app:

```
http://localhost:5000/  
http://localhost:5000/metrics
```

Create Prometheus Configuration

Create prometheus.yml:

```
global:  
  scrape_interval: 5s  
  
scrape_configs:  
  - job_name: 'monitored-app'  
    static_configs:  
      - targets: ['monitored-app:5000']
```

Set Up Docker Compose Stack

```
# docker-compose.yml
```

version: '3.7'

services:

monitored-app:

build: .

ports:

- "5000:5000"

prometheus:

image: prom/prometheus

volumes:

- ./prometheus.yml:/etc/prometheus/prometheus.yml

command:

- '--config.file=/etc/prometheus/prometheus.yml'

ports:

- "9090:9090"

grafana:

image: grafana/grafana

ports:

- "3000:3000"

Run the stack:

`docker-compose up -d`

Add Prometheus Data Source in Grafana

1. Go to **Grafana** → **Settings** → **Data Sources** → **Add Prometheus**
2. URL: `http://prometheus:9090`
3. Save & Test → “Data source is working”

Create a Grafana Dashboard

- Add panel → Query: `app_requests_total`
- Visualization: Line chart
- Title: “Total Requests Over Time”
- Save dashboard as **Application Monitoring Dashboard**

Configure Alerts in Prometheus

Add alert rules file `alert.rules.yml`:

groups:

- name: app-alerts

rules:

- alert: HighRequestRate
 - expr: rate(app_requests_total[1m]) > 10
 - for: 30s
 - labels:
 - severity: warning
 - annotations:
 - summary: "High request rate detected"
 - description: "Request rate exceeded 10 requests per minute"

Update prometheus.yml to include alert rules:

rule_files:

- "alert.rules.yml"

Restart Prometheus:

docker-compose restart prometheus

Test alert by sending multiple requests:

for i in {1..50}; do curl -s http://localhost:5000/ > /dev/null; done

Simulate Incident Response

Simulate application failure:

curl http://localhost:5000/simulate_error

Observe in Grafana:

- Spike in latency or error rate
- Alert fired in Prometheus/Alertmanager

Incident Response Actions:

1. Identify source (check docker logs monitored-app)
2. Contain the issue (restart container)
3. Document response steps
4. Update alert rules / thresholds if needed

Output:

Tool	Observation
Prometheus	Metrics collection working
Grafana	Dashboard displaying real-time request counts
Alertmanager	Alerts triggered on threshold breach
Application Logs	Shows simulated error for incident response

Result:

Successfully set up a continuous monitoring system for both application and infrastructure using Prometheus and Grafana.

Exercise 13: Implement security monitoring using cloud-native tools (AWS GuardDuty or Azure Security Center)

Aim:

To implement security monitoring on cloud infrastructure using AWS GuardDuty or Azure Security Center (Defender for Cloud) to automatically detect and respond to security threats.

Procedure:

Option 1:

Step 1: Enable GuardDuty

Step 2: Verify Data Sources

Step 3: Simulate a Threat Event (Malicious Activity)

Step 4: Review Findings

Step 5: Automate Alerting (Optional)

Step 6: Automate Incident Response

Step 7: Cleanup

Option 2: Azure Defender for Cloud Implementation

Step 1: Enable Microsoft Defender for Cloud

Step 2: Configure Security Policies

Step 3: View Security Recommendations

Step 4: Simulate a Threat Event

Step 5: Analyze Alerts

Step 7: Visualize Security Posture

Coding:

Enable GuardDuty

Using AWS Console:

1. Sign in to AWS Console → Navigate to **GuardDuty** service.
2. Click “**Enable GuardDuty**”.
3. GuardDuty automatically starts monitoring across your account.

Output: GuardDuty dashboard shows “GuardDuty is now active”.

Verify Data Sources

GuardDuty uses:

- **CloudTrail:** API call monitoring
- **VPC Flow Logs:** Network connections
- **DNS Logs:** Domain name queries

Ensure these are **enabled** in your region.

```
aws guardduty list-detectors
```

Output:

```
{
  "DetectorIds": ["123abc456def789ghi012jkl345mno678"]
}
```

Simulate a Threat Event (Malicious Activity)

AWS provides a safe sample finding generator:

```
DETECTOR_ID=$(aws guardduty list-detectors --query 'DetectorIds[0]' --output text)
aws guardduty create-sample-findings --detector-id $DETECTOR_ID
```

Output:

```
{
  "unprocessedIds": []
}
```

This generates fake threat data (e.g., “EC2 instance communicating with known malicious IP”).

Review Findings

In AWS Console → GuardDuty → Findings

Title: UnauthorizedAccess:EC2/MaliciousIPCaller

Resource: i-08ab1234c56def789

Severity: High

Each finding includes:

- Resource details (EC2, IAM, S3, etc.)
- Attack type
- Recommended remediation

Automate Alerting (Optional)

Integrate with **SNS (Simple Notification Service)**:

```
aws sns create-topic --name GuardDutyAlerts
```

```
aws sns subscribe --topic-arn <ARN> --protocol email --notification-endpoint  
you@example.com
```

Then configure GuardDuty → **Settings** → **Findings Export** → Send to SNS topic.

Output:

Email notifications for new security findings.

Automate Incident Response

Use **AWS Lambda** to automatically isolate an EC2 instance:

```
import boto3
```

```
def lambda_handler(event, context):
```

```
    ec2 = boto3.client('ec2')
```

```
    instance_id = event['detail']['resource']['instanceDetails']['instanceId']
```

```
    ec2.modify_instance_attribute(InstanceId=instance_id, Groups=['sg-isolate'])
```

```
    print(f"Isolated {instance_id} from network")
```

This triggers automatically when GuardDuty detects an attack.

Cleanup

```
aws guardduty delete-detector --detector-id $DETECTOR_ID
```

Option 2: Azure Defender for Cloud Implementation

Enable Microsoft Defender for Cloud

1. Log in to **Azure Portal** → Search **Defender for Cloud**.
2. Click **Upgrade** to enable Defender plans for your subscription.
3. Defender starts collecting data from:
 - Virtual Machines (via Log Analytics Agent)
 - Storage Accounts
 - App Services

Output:

Security score appears on the dashboard.

Configure Security Policies

In **Environment Settings** → **Security Policies**:

- Enable **Defender for Servers, App Service, and Storage**.
- Choose automatic provisioning of the Log Analytics Agent.

View Security Recommendations

Go to **Defender for Cloud** → **Recommendations**

- Enable endpoint protection on VM
- Secure management ports (RDP/SSH)
- Apply system updates

Each recommendation improves your security posture.

Simulate a Threat Event

To test detection:

1. SSH into a monitored VM.
2. Run a simulated suspicious command:

```
curl http://testmaliciousdomain.com
```

3. Wait a few minutes → Defender detects unusual outbound activity.

Expected Alert:

Alert: Suspicious outbound connection detected

Severity: Medium

Affected Resource: VM-Test1

Analyze Alerts

Go to **Defender for Cloud** → **Security Alerts**

- ☐ Alert details (timestamp, resource, IP, behavior)
- ☐ Suggested remediation actions

Example:

Title: Suspicious process execution

Resource: myVM

Recommendation: Isolate VM and run Defender Antivirus scan

Visualize Security Posture

Defender for Cloud Dashboard:

- **Secure Score:** 0–100%
- **Top Recommendations**
- **Active Alerts**

Output:

Secure Score: 74%

Recommendations: 5

Active Alerts: 1

Output:

Step	Observation	Tool
Threat detection enabled	GuardDuty / Defender active	Console
Sample findings created	Malicious IP access simulated	AWS CLI
Alerts displayed	High severity alerts generated	Portal
Automated response	Email / Lambda / Logic App triggered	AWS / Azure
Dashboard view	Security score & alerts visible	Grafana-like dashboards

Result:

Successfully implemented security monitoring using cloud-native tools (AWS GuardDuty or Azure Defender for Cloud) and verified that threats were detected and alerts generated in real time.